

Project Design Phase Solution Architecture

Date	30 September 2026
Team ID	SWUID2026286057
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	4 Marks

Solution Architecture:

This document describes the solution architecture for the ServiceNow Administrator project “Implement Client Script & UI Policy (Incident)”. The solution improves the default Incident form by adding browser-side logic (Client Scripts and UI Policies) that guides users, validates data and controls field behaviour before the record is sent to the server. The architecture connects the business problem (manual entry errors, incomplete incident submissions and slow resolution) to the ServiceNow platform features used to solve it.

1. Objectives of the Architecture

- **Best technical solution:** use native ServiceNow client-side features (Client Scripts and UI Policies) on the Incident table, with no extra plugins or licences.
- **Structure and behaviour:** describe the layers, components and data flow so that all stakeholders understand how the form behaves (Sections 2 to 4).
- **Features, phases and requirements:** define the rule set to be built (Section 5) and the delivery phases: design, configuration, testing and deployment.
- **Specifications for delivery:** build in the Development instance, capture changes in an Update Set, verify in Test and promote to Production.

2. Solution Architecture Diagram

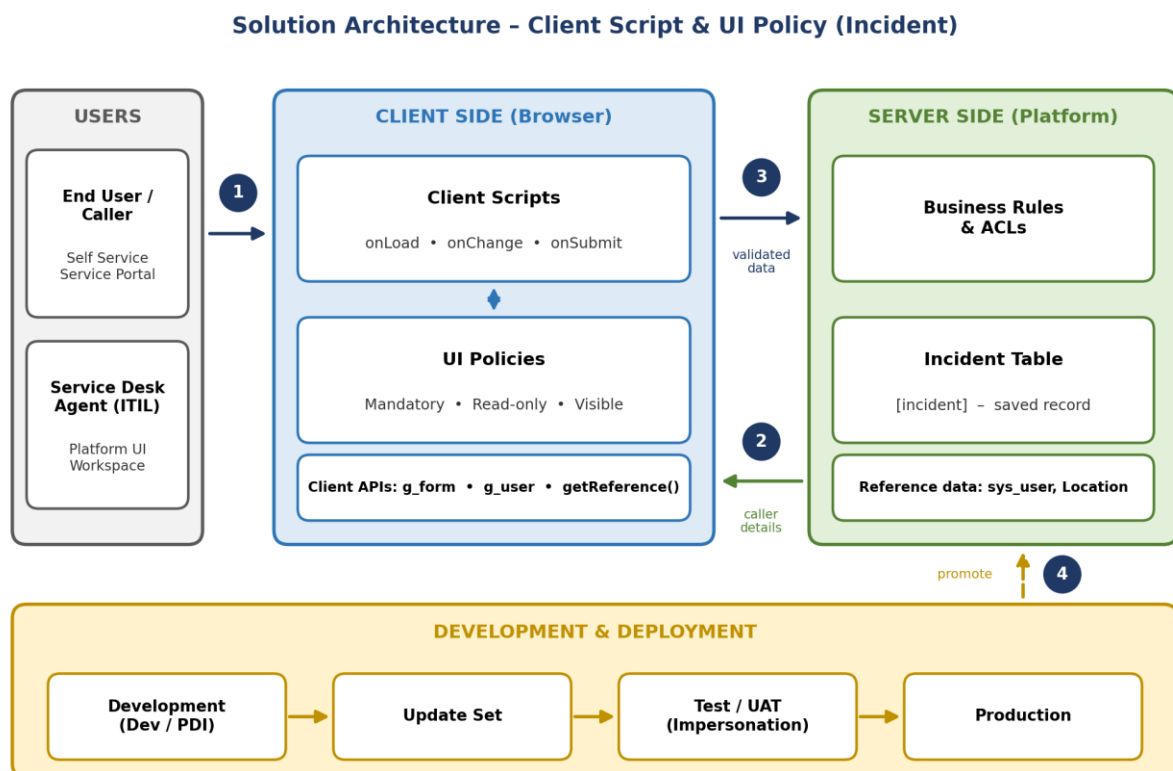


Figure 1: Architecture and data flow of the Incident Client Script & UI Policy solution

3. Architecture Components

Layer	Component	Responsibility
Users	End User / Caller; Service Desk Agent (ITIL)	Create and update incidents through Self Service / Service Portal or the Platform UI / Workspace. The form behaves according to the user's role.
Client side: Client Scripts	onLoad, onChange, onSubmit	JavaScript that runs in the browser to set defaults, show guidance, auto-populate fields and validate data before submission.
Client side: UI Policies	Mandatory, Read-only, Visible	Condition-based, declarative control of field behaviour. "Reverse if false" restores the normal state when the condition no longer applies.
Client APIs	g_form, g_user, getReference()	g_form reads and sets field values and shows messages; g_user checks the user's roles; getReference() fetches caller details without blocking the form.
Server side	Business Rules and ACLs	Existing platform logic that enforces security and data integrity on the server, independent of the client-side rules.
Data	Incident table [incident]; reference data (sys_user, Location)	Stores the saved incident record and supplies caller and location details used for auto-population.
Deployment	Dev / PDI, Update Set, Test / UAT, Production	Changes are built in Development, captured in an Update Set, tested by impersonating different roles and then promoted to Production.

4. Data Flow (numbers match Figure 1)

1. **Open form:** The user opens a new or existing incident. onLoad Client Scripts run and UI Policies are evaluated, so defaults, guidance messages and the mandatory / read-only / visible state of each field are applied.
2. **Interact and look up data:** As the user fills in the form, onChange Client Scripts and UI Policies react to each change (for example Category to Subcategory, State to resolution fields). When a caller is selected, getReference() retrieves the caller's details from the reference data and fills the related fields.
3. **Validate and submit:** On Save / Submit, the onSubmit Client Script checks the data. If a rule fails, an error message is shown and submission is stopped. If all rules pass, the validated data is sent to the server, where Business Rules and ACLs are applied and the record is saved in the Incident table.
4. **Deploy:** The configured rules are promoted from Development through an Update Set to Test / UAT and then to Production after successful testing.

5. Proposed Rule Set

No	Rule	Type	Trigger / Condition	Purpose
1	Default values and guidance	Client Script: onLoad	New incident is opened	Sets defaults (e.g. Contact type) and shows a guidance message.
2	Category and Subcategory handling	Client Script: onChange	Category changes	Clears Subcategory and shows a hint for correct selection.

No	Rule	Type	Trigger / Condition	Purpose
3	Caller auto-populate	Client Script: onChange	Caller changes	Fetches caller details with getReference() and alerts the agent for a VIP caller.
4	Final validation	Client Script: onSubmit	Save / Submit	Checks Short description length, resolution details when Resolved and hold reason when On Hold; blocks submission with an error message.
5	Core mandatory fields	UI Policy: Mandatory	Always	Makes Short description, Category, Impact and Urgency mandatory.
6	Resolution details	UI Policy: Mandatory + Visible	State is Resolved or Closed	Shows and requires Close code and Close notes.
7	On hold reason	UI Policy: Mandatory + Visible	State is On Hold	Shows and requires the On hold reason.
8	Protected fields	UI Policy: Read-only	Number, Priority, Caller (after creation); all key fields when Closed	Prevents accidental or unauthorised edits.

6. Key Design Considerations

Consideration	Approach
Performance	Rules run in the user's browser, so there is no page reload and no extra server load. Lookups with getReference() use a callback so the form is never blocked.
Security and data integrity	Client-side rules improve usability and guide users, but they are not a security control. ACLs and server-side Business Rules remain responsible for enforcing access and data integrity.
Maintainability	UI Policies are used wherever a simple condition is enough; Client Scripts are used only for logic that needs code. Clear names, conditions and execution order are used for every rule.
Testing	Each rule has test cases. Testing is done by impersonating a caller, an ITIL agent and an administrator, in both the Platform UI and Self Service / Service Portal.
Scalability	The same pattern can be reused for Problem, Change and Service Catalog (Catalog Client Scripts and Catalog UI Policies) with only field names and conditions changed.
Upgrade safety	Only standard ServiceNow features are used, with no core code changes, so the solution is easy to upgrade and to move between instances.